CONSOLIDATED_UNFINISHED_WORK_AND_P

I'll synthesize the available evidence into the consolidated report. Note: only 4 of 16 assessments returned data (the rest were rate-limited), so I'll be explicit about coverage gaps rather than inventing findings.

HelixCode — Consolidated Unfinished-Work Report & Phased Implementation Plan

> **Coverage honesty notice:** Of the 16 commissioned assessments, **only 4 returned evidence** (helix_code inner app, root meta-repo, helix_qa, containers). Assessments 4, 6–16 returned `API Error … Rate limited` and produced **no data**. Modules NOT covered by any returning assessment — `challenges`, `security`, `assets`, `github_pages_website`, `dependencies/*` (LLama_CPP, Ollama, HuggingFace_Hub), `cli_agents`, `cli_agents_resources` — are marked **NOT ASSESSED** below and must not be assumed clean or broken. Every "unverified" label from the source assessments is preserved verbatim.

---

1. Executive summary

Of the four assessed Go modules, **all four build cleanly**: `helix_code` (inner app, `go build ./...` exit 0, only cosmetic `-lobjc` cgo linker warnings), the root meta-repo (`go build ./internal/... ./cmd/... ./tests/...` exit 0), `helix_qa` (first-party `cmd/pkg/internal` exit 0), and `containers` (`go build ./...` exit 0). `go vet` was clean across all four. The most serious *functional* defects are concentrated, not pervasive: the **root meta-repo's only binary is a disabled stub** (`cmd/security-test/main.go` prints "SECURITY TESTING TEMPORARILY DISABLED"), and **helix_qa's** `list --banks banks` **command hard-fails** on a data-consistency bug (a stray `title:` instead of `name:` in `banks/atmosphere.yaml` aborts loading the whole directory). The biggest **anti-bluff (CONST-035) risks** are documentation-vs-reality gaps: the root meta-repo carries **106 root** `.md` **files** (~95 stale planning/"COMPLETION"/"CERTIFICATE" docs that contradict each other) and `helix_code` carries **84 root** `.md` **"COMPLETION" docs** asserting completeness that directly conflicts with live stubs (FAISS simulation, single-node-only consensus, six error-stub providers, Windows/macOS capture stubs). Real functional gaps in the inner app include `internal/memory/providers/faiss_provider.go` (a FAISS provider that loads no FAISS — brute-force "simulation"), `internal/worker/consensus.go:212` (multi-peer vote transport unimplemented — consensus non-functional past one node), and `internal/providers/ai_integration.go:1771+` (Cohere/HuggingFace/Mistral-cloud/CharacterAI/Replika/Anima are error stubs — though beyond the CONST-039 required set). Test coverage is **structurally hollow in the root meta-repo** (2 no-assertion `t.Log` stubs guarding ~2900 LOC, a stale `coverage.out` referencing a deleted `Example_Projects/Plandex` tree) while the inner app and helix_qa have large test corpora (767 and 359 test files respectively) that *compile* but were **not executed** here (infra-bound; `-race` not run). Tooling is uneven: `helix_code` has `.snyk` + SonarQube config (with a name mismatch: config expects `coverage.out`, repo commits `unit_coverage.out`), while **helix_qa and the root meta-repo have no** `.snyk`**/SonarQube/govulncheck wiring at all**, and `containers` lacks Snyk/Sonar/govulncheck. Concurrency was statically clean (`go vet` reported no race diagnostics) but

`-race` **was not executed in any module — race status is unverified across the board**. Net: the codebase compiles and is far from "broken," but it is **over-documented as finished** relative to several live simulation/stub paths, has two concrete user-facing breakages to fix first, and has not been dynamically test-verified in this pass.

---

2. Broken / failing / disabled (FIX-FIRST)

Module	What	Evidence	Suggested owning stream
root meta-repo	Sole binary is a disabled stub — does no work	`cmd/security-test/main.go:11` prints "⚠ SECURITY TESTING TEMPORARILY DISABLED"; line 18 `// Placeholder for future security testing` (Assessment 2)	S1-Root-Binary
helix_qa	`list --banks banks` hard-fails, aborting load of entire bank directory	`./helixqa list --banks banks` → `error: bank file banks/atmosphere.yaml: test case 167: CME-KA-AUDIO-MATRIX-001 missing name`; root cause: 7 cases use `title:` not `name:` (file has 169 `name:` vs 7 `title:`); a single valid bank loads fine (Assessment 3)	S2-QA-BankLoader
root meta-repo	Makefile `scan-*/verify-foundation` target `-C HelixCode` (uppercase) — breaks on case-sensitive Linux hosts	Makefile uses `$(MAKE) -C HelixCode`; actual dir is `helix_code`; resolves on macOS `core.ignorecase=true` only (Assessment 2)	S1-Root-Binary
root meta-repo	`verify-foundation` references a missing `bluff-detector.sh` and is documented as known-failing	`scripts/bluff-detector.sh` missing (Makefile skips with "Phase 4 deliverable"); Makefile comment: `verify-foundation` "exits 1 because verify-llmsverifier-pin-parity reports the known LLMsVerifier dual-pin divergence" (Assessment 2)	S1-Root-Binary
root meta-repo	`tests/e2e/core/{simple,auth_simple}_test.go` are no-assertion `t.Log` stubs masquerading as e2e tests (§11.4 PASS-bluff)	`TestSimple` body = `t.Log("Simple test")`; `TestAuthSimple` body = `t.Log("Auth simple test")` (Assessment 2)	S1-Root-Binary
helix_code	i18n command tests convert likely-real failures into silent SKIPs (anti-bluff smell)	`cmd/helix_config/main_i18n_test.go:159,314,451` `t.Skipf("... short-circuited before printing — config.SaveHelixConfig likely failed")` — 48 total non-SKIP-OK, non-`Short()` skips (Assessment 1)	S3-Inner-AntiBluff

---

3. Dead code (whole features unconnected)

Module	Suspect (path)	Why suspected	Verification needed
root meta-repo	`internal/security` (913+613 LOC), `internal/fix` (885 LOC), `internal/testing` (567 LOC), `internal/theme/theme.go`	~2965 LOC reachable from no runnable binary; `cmd/security-test/main.go` (only `main`) imports none; only intra-cluster imports exist (Assessment 2)	Decide: wire to the (re-enabled) security-test binary, or delete
root meta-repo	`coverage.out`	Stale: 60/61 lines reference deleted `dev.helix.code/Example_Projects/Plandex/...` tree; also CONST-053 build-artifact-in-VCS concern (Assessment 2)	Regenerate or remove + gitignore
helix_code	`demo_cross_provider_sharing.go`, `demo_model_management.go`, `simple_test_runner.go`, `test_model_management.go` (root `package main`)	Dead `package main` funcs, 0 callers, wired to no entrypoint (only `main.go` has `func main`) (Assessment 1)	Delete or wire to a real command
helix_code	`internal/server` `notImplemented` handler	Referenced only in `handlers_test.go:941`; 0 production-route refs — dead helper (Assessment 1)	Confirm no route intends to use it; delete
helix_code	`internal/tools/lsp_fakeserver/main.go`	"fake server" in production `internal/` tree, returns `"method not implemented by fake server"` (line 155) — placement violates no-mocks-in-prod unless test-only infra (Assessment 1)	Confirm test-only; relocate out of `internal/` or document
containers	`pkg/lazyservice/orchestrator.go`	Strongest suspect: **0 importers AND 0 test files**; unwired and untested (Assessment 5)	Wire a consumer + add tests, or remove
containers	`pkg/orchestrator`, `pkg/policy`, `pkg/serviceregistry`, `pkg/crossbuild`, `pkg/brokertest`	0 internal/cmd/helix_code/challenges importers; tested-only library surface awaiting consumers; `crossbuild` "may be exercised only via challenges scripts — unverified" (Assessment 5)	Trace challenges-script usage; wire or document as public API
helix_qa	`cmd/helixqa-*` capture/vision binaries (`helixqa-lpips`, `helixqa-omniparser`, etc.)	"**unverified** whether every binary is invoked by the orchestrator vs operator-run standalone" (Assessment 3)	Full import-graph trace to classify

Module	Suspect (path)	Why suspected	Verification needed
			standalone vs wired

---

4. Test-coverage gaps by module & test-type

Legend: present (P) / missing or not-evidenced (—) / unverified-present (?). “Present” means test files exist and compile; **none were executed in this pass.**

helix_code (inner app) — 767 test files; `go vet` clean. - unit: P (compile-clean) · integration: ? (`-tags=integration`, not run) · e2e: ? (`tests/e2e/challenges/cmd/runner`, not run) · security: ? (`tests/security/`, not run) · stress: ? (Makefile `stress-chaos` targets exist) · chaos: ? · performance/benchmark: ? (`tests/performance/`) · UI/UX: — (Fyne/tview apps present, no evidenced UI test) · Challenges: ? · helix_qa: ? — **Gap: dynamic execution + `-race` never run; UI/UX automation unevidenced.**

root meta-repo — only 2 test files, both no-assertion stubs. - unit: — (zero real unit tests over ~2900 LOC of `internal/`) · integration: — · e2e: — (`tests/e2e/core/*` are `t.Log` stubs) · all other types: — · Challenges: P (shell challenges with proper SKIP-OK markers, `tests/e2e/challenges/*.sh`) — **Gap: near-total — the dead security/fix/testing cluster has no real tests of any type.**

helix_qa — 359 first-party test files; `go vet` clean; 0 bare skips. - unit: P · integration: ? · e2e: ? · security/stress/chaos/perf/benchmark: ? (not run) · UI/UX: ? (vision/capture binaries) · Challenges/helix_qa: P (this IS the QA platform) — **Gap: Windows/macOS capture paths are stubs (see §2/§8), so any “complete” claim on those platforms is unevidenced; `-race` not run.**

containers — 161 test files; test packages compile. - unit: P · integration: ? · e2e: ? · security: — (no evidenced suite) · stress/chaos: ? (Makefile `anti-bluff-mutation`, `test-race`) · performance/benchmark: — · UI/UX: n/a · Challenges/helix_qa: ? — **Gaps: `pkg/lazyservice` has zero tests; orphan packages tested-only; no security/perf suites evidenced.**

NOT ASSESSED (no test-coverage data): challenges, security, assets, github_pages_website, dependencies/*, cli_agents, cli_agents_resources.

---

5. Tooling gaps (Snyk / SonarQube / govulncheck / race / profiling)

Project bans hosted CI (Rule 1). Close each gap with a containerized run via the `./helix` facade / `containers` submodule `pkg/boot`+`pkg/compose` (§11.4.76), invoked from Makefile/script targets — never hosted pipelines, never hand-run `docker`/`podman`.

Module	Snyk	SonarQube	govulncheck	`-race`	profiling
helix_code	P (`.snyk`)	P (`sonar-project.properties`)	referenced in Makefile + `scripts/`	referenced (run_all_tests.sh:127, Makefile stress/	

Module	Snyk	SonarQube	govulncheck	-race	profiling
		but coverage.out **vs committed** unit_coverage.out **name mismatch — scan success unverified**	run-all-tests.sh	chaos) but **not executed here**	internal/pprofutil pkg exists
root meta-repo	— (none at root)	— (none at root)	only in scripts/release-gate-test.sh	only in release-gate/ summary scripts	—
helix_qa	—	—	— (not in Makefile/ scripts)	P (Makefile test-race) but not run	—
containers	—	—	—	P (Makefile test-race, scripts/anti-bluff/lib/go.sh)	—

Cross-cutting: -race and govulncheck were **not executed in any module** during assessment — every "race-clean" statement is static-vet-only and **unverified**. NOT ASSESSED modules have no tooling data.

---

6. Concurrency-safety risks

All entries are **static suspicions; none runtime-confirmed** (-race not run anywhere).

Module	Area/file:func	Suspected issue	Fix direction
helix_code	internal/worker/consensus.go:212 (election path)	Candidate state never resolves when vote-request transport "is NOT implemented" — logic dead-loop, not a data race	Implement multi-peer vote transport; add bounded election timeout + non-blocking state machine
helix_code			

Module	Area/file:func	Suspected issue	Fix direction
	`internal/tools/lsp_fakeserver/main.go`	Long-lived jsonrpc2 server living in production tree	Relocate to test-only infra; bound lifecycle
helix_code	31 `time.Sleep` in prod `internal/cmd`; 40 `go func()` launches	Sleeps in prod paths = timing/blocking suspects; 1044 `.Lock()` calls not audited for missing `defer Unlock()`	Replace fixed sleeps with context-cancellable waits; audit each `Lock` for paired `defer Unlock`
containers	`pkg/monitor/system.go:51` (`Sleep 50ms`), `pkg/health/helix_infra.go:72` (`Sleep 1s`) in poll/wait loops	Potential blocking if not context-cancellable	Make poll loops context-aware; non-blocking select on ctx.Done()
containers	`cmd/distributed-test/main.go:341` (`Sleep 5s`)	Fixed sleep in CLI flow → flakiness/timing dependence	Replace with readiness probe / poll-until-ready
containers	21 `context.Background()` in lib/cmd	Ungoverned cancellation — caller cannot time out	Thread caller-supplied `ctx` with deadlines
containers	`pkg/lazyservice` sync-based on-demand start	Locking unproven — exercised by no caller	Add integration test exercising lazy-init under contention
helix_qa	`pkg/capture/windows_capture.go:readFrames`	No-op goroutine → silent hang / missing output on Windows (not a race)	Implement frame reading; or fail-fast with honest error

7. Docs / user manuals / video courses / website / diagrams / SQL gaps

- **Severe doc bloat + anti-bluff contradiction (root meta-repo): 106 root `.md` files** — the cascaded governance set (`CLAUDE.md` 193KB, `AGENTS.md` 205KB, `CRUSH.md`, `QWEN.md`, `CONSTITUTION.md`) plus **~95 stale planning docs** all timestamped `Jun 3 14:40` (single bulk import). Multiple competing "FINAL"/"COMPLETION"/"COMPREHENSIVE" reports contradict each other. `README.md` claims "Version 1.0.0 / Phase 1 Completed" with feature lists (DB schema, JWT auth) that **live in the inner submodule, not this meta-repo** — README does not describe the meta-repo's actual content (the dead security cluster). Committed binary exports `README.html` (157KB), `README.pdf` (211KB). **Action: consolidate the ~95 planning docs into one authoritative doc; rewrite root README to describe the meta-repo's real contents.** (Assessment 2)
- **Completion docs overstate state (helix_code): 84 root `.md`** "COMPLETION"/"CERTIFICATE" docs (`PROJECT_COMPLETION_CERTIFICATE.md`, `FINAL_COMPLETION_*`, `EXECUTIVE_TEST_SUMMARY.md`) assert completeness that conflicts with live stubs (FAISS sim, consensus stub, six error-stub providers). CONST-035 / Rule 9 risk. Committed `README.pdf`. **Action: reconcile docs to actual capability; demote false-finished claims.** (Assessment 1)
- **Completion docs vs live stubs (helix_qa):** many `*_COMPLETE.md` / `PROJECT_COMPLETE.md` "do not match" the live Windows-capture/screenshot stubs and the bank-loader bug — "completion docs overstate state (unverified beyond the specific stubs cited)." (Assessment 3)
- **Diagrams/SQL/video courses:** root meta-repo carries `VIDEO_COURSE_CURRICULUM.md` and `WEBSITE_UPDATE_SUMMARY.md` among the stale set; `github_pages_website` and `assets` submodules are **NOT ASSESSED**. SQL schema docs were not separately evidenced. **No evidence either way — do not claim complete or missing.**
- **containers docs appear matched** for the consumed core (`pkg/boot/compose/health`), with full governance triplets (`.md`+`.html`+`.pdf`); doc coverage for orphan packages (lazyservice/orchestrator/policy) "not individually verified." (Assessment 5)

---

8. Phased implementation plan

Dependency-ordered. Phase 1 fixes what §2 *proves* broken. Every exit criterion requires captured terminal output (Rule 8/9, §11.4.5/§11.4.69). All long runs background per §11.4.89; subagent-driven per §11.4.70.

Phase 1 — Fix-first (proven broken/disabled)

- **Stream 1A — Root security-test binary** — re-enable or honestly retire `cmd/security-test/main.go` (currently a disabled stub); decide fate of the unwired `internal/{security,fix,testing}` cluster (wire-or-delete). — *owned:* root meta-repo. — *tests:* unit (real assertions) + a Challenge exercising the re-enabled binary. — *exit:* `./bin/security-test` runs and produces real output (pasted); no "DISABLED/placeholder" string remains; §11.4.115 RED-on-broken-then-GREEN captured.
- **Stream 1B — helix_qa bank-loader** — fix the 7 `title:`→`name:` cases in `banks/atmosphere.yaml` (and any siblings); make the loader resilient (report all bad cases, don't abort whole dir on one). — *owned:* helix_qa. — *tests:* unit (loader rejects/reports per-

case) + integration (`list --banks banks` lists all). — *exit:* `./helixqa list --banks banks` exits 0 listing every bank (pasted); RED test reproduces the abort on prefix data.
- **Stream 1C — Root Makefile portability + gates** — fix `-C HelixCode→helix_code`; supply or remove `scripts/bluff-detector.sh`; resolve the known-failing `verify-foundation` LLMsVerifier dual-pin divergence. — *owned:* root meta-repo. — *tests:* a containerized run of each `scan-*`/`verify-foundation` target on a case-sensitive filesystem. — *exit:* `make verify-foundation` exits 0 with bluff-detector gate active (pasted).
- **Stream 1D — Real e2e tests (root)** — replace `tests/e2e/core/{simple,auth_simple}_test.go` `t.Log` stubs with asserting tests. — *owned:* root meta-repo. — *tests:* e2e with real assertions. — *exit:* tests assert observable behavior and FAIL when the behavior is broken (mutation-proven, §1.1).

Phase 2 — Anti-bluff reconciliation (docs ? code) + silent-skip removal

- **Stream 2A — Doc consolidation (root)** — collapse ~95 stale planning `.md` into one authoritative doc; rewrite root README to describe the meta-repo's true contents; regenerate/remove stale exports. — *owned:* root. — *tests:* a sync gate (Docs Chain §11.4.106). — *exit:* one tracked authoritative doc; README matches `internal/` reality; CM-SUMMARY-CLARITY gate passes.
- **Stream 2B — Completion-doc reconciliation (helix_code + helix_qa)** — demote false-finished claims in the 84 + N "COMPLETION"/"CERTIFICATE" docs to match live stubs. — *owned:* helix_code, helix_qa. — *tests:* anti-bluff grep gate. — *exit:* no doc claims a stubbed feature complete (pasted grep).
- **Stream 2C — Silent-skip removal (helix_code)** — convert the 48 non-SKIP-OK skips (esp. `cmd/helix_config/main_i18n_test.go:159/314/451`) into real failures or SKIP-OK-tagged honest skips. — *owned:* helix_code. — *tests:* the skips become assertions. — *exit:* `make no-silent-skips` exits 0 (pasted).
- **Stream 2D — Hygiene (root)** — remove stale `coverage.out` (deleted-Plandex refs) + gitignore it; fix helix_code `coverage.out?unit_coverage.out` SonarQube name mismatch. — *owned:* root, helix_code. — *exit:* CONST-053 audit clean; SonarQube scan runs (pasted).

Phase 3 — Functional stub completion

- **Stream 3A — Memory/FAISS** — implement real FAISS/IVF/GPU indexing in `internal/memory/providers/faiss_provider.go` (or rename + document as brute-force, removing "simulation"). — *owned:* helix_code. — *tests:* integration vs real index. — *exit:* config fields (`IndexType`/`NList`/`NProbe`) actually used; benchmark captured.
- **Stream 3B — Worker consensus** — implement multi-peer vote-request transport (`internal/worker/consensus.go:212`). — *owned:* helix_code. — *tests:* integration (3+ node election) + chaos (process-death). — *exit:* leader elected across peers with captured evidence.
- **Stream 3C — Capture stubs (helix_qa)** — implement `pkg/capture/windows_capture.go:108` frame reading, `pkg/screenshot/windows_engine.go:46` real read-back, `pkg/capture/macos_capture.go:313` ScreenCaptureKit (or honest SKIP per §11.4.81). — *owned:* helix_qa. — *tests:* per-OS captured-evidence (§11.4.81/§11.4.107 liveness). — *exit:* real frames captured per platform OR honest gap-cited SKIP.
- **Stream 3D — Provider stubs (helix_code)** — implement or explicitly classify the six error-stub providers in `internal/providers/ai_integration.go:1771+`

(note: beyond CONST-039 required set — may be legitimately deferred). — *owned:* helix_code. — *exit:* each provider either makes real calls or is documented as won't-fix/deferred per §11.4.112.

Phase 4 — Tooling, dead-code resolution, dynamic verification

- **Stream 4A — Containerized scans** — add `.snyk` + SonarQube + `govulncheck` + `-race` containerized targets to helix_qa, containers, and root (§11.4.76). — *owned:* all four. — *exit:* each scan runs in-container with output captured.
- **Stream 4B — Dead-code disposition** — wire-or-delete `containers/pkg/lazyservice` (+ orphan pkgs), helix_code root demo orphans, `internal/server notImplemented`, `lsp_fakeserver`. — *owned:* containers, helix_code. — *exit:* import-graph proof no orphan remains, or each documented as public API.
- **Stream 4C — Dynamic test execution** — actually run unit + integration + e2e + `-race` across all four modules via `make test-infra-up`. — *owned:* all four. — *exit:* pasted pass/fail with `-race`; current "compile-only/unverified" labels resolved.
- **Stream 4D — NOT-ASSESSED module sweep** — commission fresh assessments for challenges, security, assets, github_pages_website, dependencies/*, cli_agents (rate-limited this round).* — owned:* orchestrator. — *exit:* evidence-backed report per module.

---

9. Prioritized first actions (top 10)

1. **Fix** `banks/atmosphere.yaml` `title:`→`name:` **(7 cases) + make loader non-fatal** — helix_qa — a core QA command (`list --banks banks`) is hard-broken today, blocking bank-driven QA. (Assessment 3)
2. **Re-enable or honestly retire** `cmd/security-test/main.go` — root meta-repo — the meta-repo's only binary currently does nothing; pure §11.4 PASS-bluff surface. (Assessment 2)
3. **Fix Makefile** `-C HelixCode`→`helix_code` — root meta-repo — `scan-*`/`verify-foundation` silently break on any case-sensitive (Linux) host. (Assessment 2)
4. **Replace** `tests/e2e/core/*` `t.Log` **stubs with asserting tests** — root meta-repo — these PASS while asserting nothing; canonical anti-bluff defect. (Assessment 2)
5. **Convert 48 silent skips (esp.** `main_i18n_test.go:159/314/451`**)** — helix_code — they hide likely-real failures behind SKIP. (Assessment 1)
6. **Remove stale** `coverage.out` **+ fix** `coverage.out/unit_coverage.out` **Sonar mismatch** — root + helix_code — stale artifact references a deleted tree; Sonar scan success is unverified. (Assessments 1, 2)
7. **Decide fate of root** `internal/{security,fix,testing}` **cluster (~2965 LOC)** — root meta-repo — dead-or-feature; either wire to the re-enabled binary or delete. (Assessment 2)
8. **Run** `-race` **+** `govulncheck` **containerized across the 4 assessed modules** — all — race/vuln status is currently *unverified* everywhere. (Assessments 1–3, 5)
9. **Resolve** `containers/pkg/lazyservice` **(unwired AND untested)** — containers — strongest dead-code suspect; wire+test or delete. (Assessment 5)
10. **Reconcile root + helix_code completion docs to live stubs** — root, helix_code — 106 + 84 `.md` over-claim "complete" against real simulation/stub paths (CONST-035 / Rule 9). (Assessments 1, 2)

Reminder for the next wave: before acting, re-commission the 12 rate-limited assessments (§8 Stream 4D) — `challenges`, `security`, `assets`, `github_pages_website`, `dependencies/*`, `cli_agents*` have **zero evidence** in this report and must not be treated as either clean or broken. ## Sources

verified Sources verified 2026-06-09: internal HelixCode consolidation/planning document — no third-party-service instructions; cross-referenced against project state (docs/Issues.md, docs/Fixed.md, .gitmodules, docs/CONTINUATION.md) as of 2026-06-09.